

Mini Project Report
of
Voice-Controlled Robot Car:
An IoT Solution

Submitted in Fulfilment of the Course
EE-427 Computer Networks

Submitted by

Suyash D Nahar 231EE257
Thakkar Vanshi Ramesh 231EE259
Tridhatri Prakash 231EE260
Venkatesh S 231EE262
Vijeeth J Poojary 231EE263

Under the Guidance of

Dr. K. P. Vittal
Professor, Department of Electrical and Electronics Engineering

To

Department of Electrical and Electronics Engineering
of
Bachelor of Technology in Electrical and Electronics Engineering



**DEPARTMENT OF ELECTRICAL AND ELECTRONICS
ENGINEERING**

**NATIONAL INSTITUTE OF TECHNOLOGY KARNATAKA
SURATHKAL, MANGALURU – 575025**

Acknowledgement

We express our profound gratitude and deep regard to **Dr. K. P. Vittal (Professor, Department of Electrical and Electronics Engineering, NITK Surathkal)** for his guidance, valuable feedback, and constant encouragement throughout this project. His insightful suggestions on network protocol design and IoT architecture were instrumental in shaping the final system.

We are also grateful to the **Department of Electrical and Electronics Engineering**, National Institute of Technology Karnataka, Surathkal, for providing the necessary resources and facilities to complete this project.

Surathkal, Mangaluru – 575 025

April 2025

Contents

Acknowledgement	i
Abstract	1
1 Introduction	2
2 Methodology	3
2.1 Software Tools Used	3
2.2 Hardware Tools Used	4
3 Network Architecture	5
3.1 System Tiers	5
3.2 OSI Layer Mapping	7
4 Implementation	8
4.1 Hardware Configuration	8
4.2 ESP32 Firmware	9
4.3 FastAPI Bridge Server	9
4.4 React Frontend	9
4.5 Keyboard Controller	10
4.6 Modes of Operation	11
5 Results and Discussion	12
5.1 Latency Measurements	12
5.2 Challenges and Solutions	12
5.3 Speed Control Results	13
6 Conclusion	14
Source Code	16

Abstract

The rapid proliferation of smart devices and the Internet of Things (IoT) has expanded the scope of remote control applications beyond local networks to internet-scale deployments. This project demonstrates an end-to-end IoT system for the Wi-Fi control of a four-wheeled robot car using an ESP32 microcontroller, with three independent control interfaces: voice commands via the browser-native Web Speech API, a graphical D-pad, and a Python keyboard controller.

Motion commands (forward, backward, left, right, stop) and speed levels (LOW, MED, HIGH) are transmitted from a React frontend hosted on Vercel through a secure FastAPI bridge server tunnelled over ngrok to the ESP32 on the local network. The architecture illustrates key computer networking concepts including HTTP/REST, TLS 1.3, CORS, API key authentication, NAT traversal via reverse tunnelling, and the full OSI protocol stack across both internet and LAN segments.

The system was successfully demonstrated with real hardware, achieving sub-300 ms round-trip latency from voice utterance to motor actuation under normal network conditions. The deployed frontend is publicly accessible at <https://ee427-robot.vercel.app>.

Chapter 1

Introduction

The Internet of Things (IoT) has become a transformative paradigm enabling physical devices to be monitored and controlled over internet-scale networks. Modern IoT deployments require the integration of embedded hardware, network protocol stacks, cloud hosting, and user interface design into a cohesive system. This project implements all four layers in the context of a remotely controlled robot car.

The core hardware is an ESP32 microcontroller, which connects to a local Wi-Fi network and hosts a lightweight HTTP server. An L298N dual H-bridge motor driver translates digital GPIO signals and PWM output from the ESP32 into mechanical motion across four DC gear motors. The system supports four directions of movement and three discrete speed levels (LOW, MED, HIGH) controlled via PWM duty cycle.

Internet connectivity is achieved through a three-tier software architecture:

1. A **React frontend** deployed on Vercel accepts user input through voice recognition or manual D-pad controls.
2. A **FastAPI bridge server** running on the local machine authenticates requests with an API key and relays commands to the ESP32 over the LAN.
3. An **ngrok reverse tunnel** provides a stable public HTTPS endpoint for the bridge server, enabling the Vercel-hosted frontend to reach the LAN device through NAT and firewalls.

This project provides practical exposure to REST API design, HTTP/HTTPS mechanics, TLS termination, CORS policy enforcement, NAT traversal, and the mapping of application-layer commands through the OSI protocol stack to physical motor actuation — directly addressing the curriculum objectives of EE-427 Computer Networks.

Chapter 2

Methodology

2.1 Software Tools Used

- **Arduino IDE**: Development environment for writing and flashing C++ firmware to the ESP32. Configures Wi-Fi station (STA) mode, instantiates the `WebServer` class on port 80, and registers eight HTTP route handlers for motion and speed commands.
- **FastAPI (Python)**: Lightweight ASGI web framework for the bridge server. Handles `X-API-Key` header authentication, validates JSON request bodies using Pydantic models, and forwards commands to the ESP32 via a persistent `requests.Session`.
- **React + Vite (TypeScript)**: Frontend framework for the control UI. Implements voice recognition using the browser-native `SpeechRecognition` API, a D-pad for manual control, and speed selector buttons (LOW / MED / HIGH).
- **Vercel**: Cloud platform hosting the React frontend as a static site with global CDN distribution. Build-time environment variables (`VITE_API_URL`, `VITE_API_KEY`) configure the API endpoint and authentication key.
- **ngrok**: Reverse tunnelling service exposing the locally running FastAPI server (port 8000) as a public HTTPS endpoint. Performs TLS termination at the edge, enabling secure communication from the internet to the LAN bridge server.
- **Web Speech API**: Browser-native speech recognition API (available in Chrome and Edge) that provides continuous transcription of voice input. Interim and final results are processed with regular-expression pattern matching; an 800 ms time-based debounce prevents duplicate command firings.
- **pynput**: Python library used in the standalone keyboard controller (`esp32_controller.py`) to capture keyboard events non-blockingly. WASD keys map to directional commands; keys 1/2/3 set speed levels, bypassing the browser and ngrok entirely.
- **uvicorn**: ASGI server used to run the FastAPI application locally on port 8000.

2.2 Hardware Tools Used

- **ESP32 Microcontroller:** Dual-core 32-bit Xtensa LX6 processor with integrated 802.11 b/g/n Wi-Fi (2.4 GHz only). Operates in station (STA) mode, connects to the local router, and hosts an HTTP server on port 80. The LEDC (LED Control) PWM peripheral generates motor speed signals.
- **L298N Motor Driver:** Dual H-bridge module capable of driving two DC motors simultaneously. Speed is regulated via PWM on the enable pins (ENA, ENB); direction is controlled via input pins (IN1–IN4). The module includes an onboard 5 V regulator for logic supply.
- **4× DC Gear Motors:** Provide mechanical drive to each wheel. Speed is proportional to PWM duty cycle: LOW = 100/255 ($\approx 39\%$), MED = 175/255 ($\approx 69\%$), HIGH = 255/255 (100%).
- **18650 Li-Ion Battery Pack (2S):** Two 3.7 V cells in series providing ≈ 7.4 V nominal to the L298N motor driver VCC input. The ESP32 is powered separately via USB during development.
- **Acrylic 4-Wheel Robot Chassis:** Transparent acrylic frame with four motor mounts housing the drive train, motor driver, battery pack, and ESP32.
- **Jumper Wires and Connectors:** Connect ESP32 GPIO pins to L298N input/en-able pins and the battery pack to the motor driver power rails.

Chapter 3

Network Architecture

The system follows a three-tier IoT architecture spanning the public internet and a private LAN, as shown in Figure 3.1.

3.1 System Tiers

Tier 1 — Edge Device (ESP32 on LAN): The ESP32 connects to the home router in station mode, assigned IP address 192.168.1.192. It hosts an HTTP server on port 80 responding to the routes listed in Table 3.1.

Route	Action
GET /F	Drive forward
GET /B	Drive backward
GET /L	Turn left
GET /R	Turn right
GET /S	Stop (PWM = 0, dynamic braking)
GET /SPD/LOW	Set speed to LOW (PWM 100/255)
GET /SPD/MED	Set speed to MED (PWM 175/255)
GET /SPD/HIGH	Set speed to HIGH (PWM 255/255)

Table 3.1: ESP32 HTTP API route table.

Tier 2 — Bridge Server (FastAPI on Mac): A FastAPI server on the same LAN machine authenticates all incoming requests via the `X-API-Key` HTTP header matched against the `BRIDGE_API_KEY` environment variable. It validates the command body using Pydantic, then forwards a `GET` request to the ESP32. A persistent `requests.Session` with configurable connect/read timeouts is used. Read timeouts from the ESP32 (which may occur when the motor command blocks the firmware’s response loop) are handled gracefully: the server returns `{"ok":true, "esp_warning":"read_timeout"}` rather than a 502 error, since the command typically executes regardless.

An ngrok agent forwards public HTTPS traffic on the ngrok subdomain to `localhost:8000`, providing TLS 1.3 termination and NAT traversal without requiring any router port-forwarding.

Tier 3 — Frontend (React on Vercel): The React application, built with Vite and TypeScript, is deployed to Vercel’s global CDN and publicly accessible at <https://ee427-robot.vercel.app>. It communicates with the bridge server exclusively over HTTPS, including the `ngrok-skip-browser-warning` header to bypass the ngrok free-tier interstitial page.

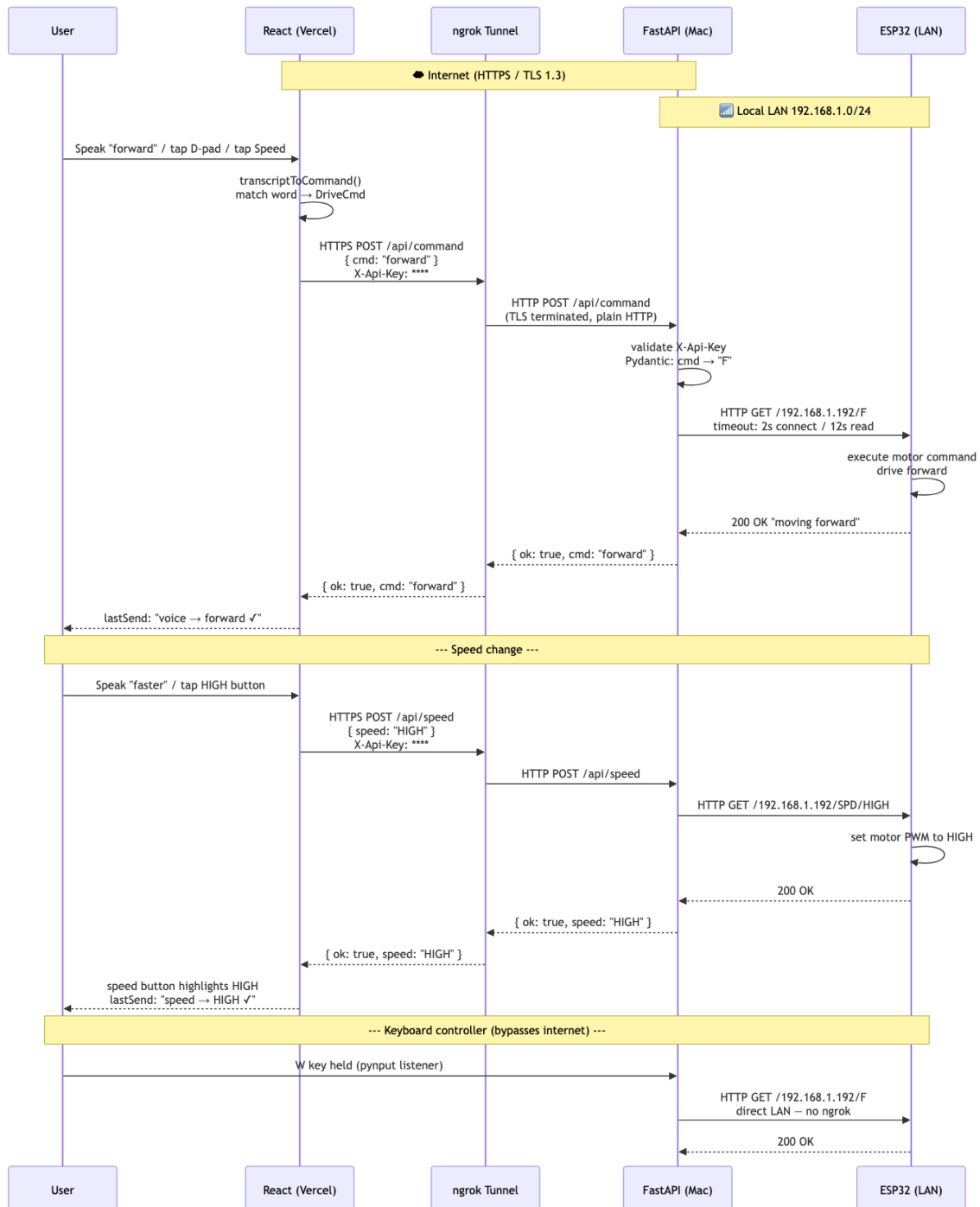


Figure 3.1: End-to-end message sequence diagram showing a motion command, a speed change, and the keyboard controller path which bypasses the internet entirely.

3.2 OSI Layer Mapping

Table 3.2 maps the command path across the OSI protocol stack for both network segments — the internet-facing path (Browser to ngrok) and the LAN path (FastAPI to ESP32).

Layer	Browser → ngrok (Internet)	FastAPI → ESP32 (LAN)
7 App	HTTPS POST /api/command JSON	HTTP GET /F, plain-text response
6 Pres	TLS 1.3 encryption; JSON serialisation	No encryption; ASCII text
5 Session	HTTP/1.1 keep-alive	<code>requests.Session</code> connection reuse
4 Trans	TCP; dst port 443	TCP; dst port 80; 2s connect timeout
3 Net-work	Internet routing via BGP; DNS lookup	192.168.1.0/24 subnet; ARP MAC resolution
2 Link	802.11 Wi-Fi / Ethernet MAC frames	802.11 2.4 GHz Wi-Fi MAC frames
1 Phys	RF / copper / fibre (ISP uplink)	2.4 GHz RF; ESP32 ceramic antenna

Table 3.2: OSI layer mapping across both network segments.

Chapter 4

Implementation

4.1 Hardware Configuration

The L298N motor driver is connected to the ESP32 as follows: direction pins IN1/IN2 drive the left motor pair; IN3/IN4 drive the right motor pair. Enable pins ENA and ENB receive PWM signals from the ESP32 LEDC peripheral (channels 0 and 1 respectively). The 7.4 V battery pack powers the L298N VCC; the onboard 5 V regulator supplies logic power to the L298N. The ESP32 GPIO outputs (3.3 V) are directly compatible with the L298N logic inputs.

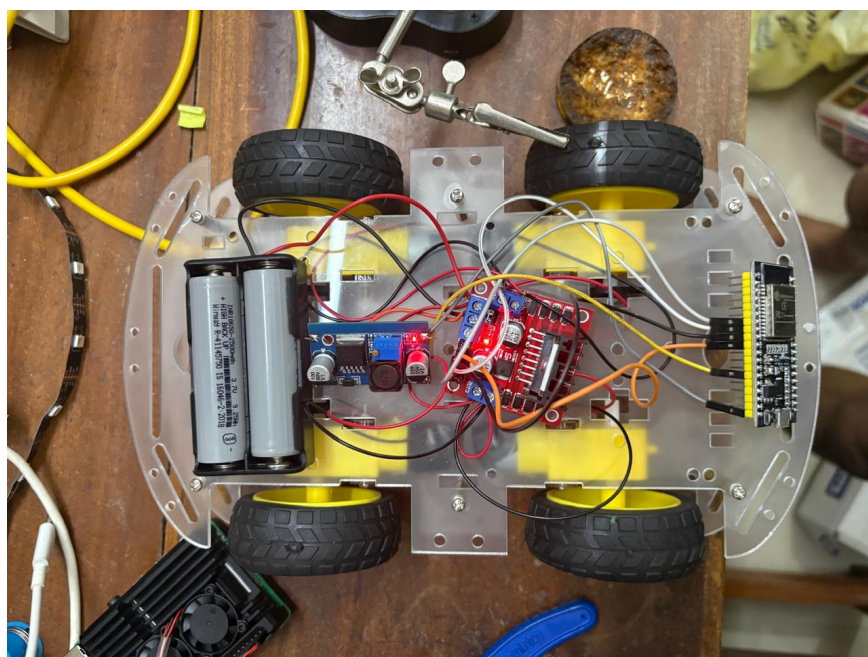


Figure 4.1: Top view of the robot car: ESP32 (right), L298N motor driver (centre, red board), and 18650 Li-Ion battery pack (left) mounted on the acrylic chassis.

4.2 ESP32 Firmware

The firmware is written in C++ using the Arduino framework for ESP32. On startup, the ESP32 connects to the configured Wi-Fi network in station mode and prints its assigned IP address to the serial console at 115200 baud. Eight route handlers are registered with the `WebServer` object before calling `server.begin()`.

Speed is maintained as a global integer `gSpeed` (0–255) updated by the `/SPD/*` route handlers. Motion handlers set the H-bridge direction pins via `digitalWrite()` and call `ledcWrite()` with the current `gSpeed` value on both PWM channels simultaneously, ensuring symmetric drive on all four wheels.

The stop handler (`/S`) drives all direction pins LOW and sets PWM to zero, applying dynamic braking by shorting the motor terminals through the H-bridge.

4.3 FastAPI Bridge Server

The bridge server exposes two authenticated POST endpoints:

- **POST `/api/command`:** Accepts JSON body `{cmd: "forward"|"backward"|"left"|"right"|"stop"}` maps the command string to an ESP32 route letter via a dictionary, and issues `GET /{letter}` to the ESP32.
- **POST `/api/speed`:** Accepts `{speed: "LOW"|"MED"|"HIGH"}` and issues `GET /SPD/{level}` to the ESP32.

CORS is configured via the `CORS_ORIGINS` environment variable (comma-separated list of allowed origins), supporting both the Vercel production URL and any preview deployment subdomains. All sensitive values are stored in a `.env` file loaded at startup via `python-dotenv`.

4.4 React Frontend

The frontend provides three control modes operating over the same API:

1. **Voice Control:** Uses the browser `SpeechRecognition` API in continuous mode with interim results enabled. Only finalised transcript segments trigger command matching via the `transcriptToCommand()` function. Matched keywords: *forward/front*, *back/backward*, *left*, *right*, *stop*, *faster/fast* (speed +1 step), *slower/slow* (speed –1 step). An 800 ms time-based debounce prevents duplicate commands from repeated recognition events.
2. **D-pad (Manual):** Five buttons in a cross arrangement send direct drive commands via `POST /api/command`. A central stop button halts the robot.
3. **Speed Control:** Three buttons (LOW / MED / HIGH) send `POST /api/speed`. The active level is visually highlighted. Voice speed commands adjust the level relatively by ± 1 step using a `useRef` mirror of the speed state, avoiding stale closure issues in the voice callback.



Figure 4.2: Live demonstration: the deployed web application running on a mobile browser controlling the robot car. Voice transcript “forward” is visible; LOW speed is active.

4.5 Keyboard Controller

A standalone Python script (`esp32_controller.py`) provides direct LAN control without requiring a browser or ngrok. Using `pynput` for non-blocking keyboard event capture:

Key	Action
W	Forward (GET /F)
S	Backward (GET /B)
A	Left (GET /L)
D	Right (GET /R)
E	Stop (GET /S)
1	Speed LOW (GET /SPD/LOW)
2	Speed MED (GET /SPD/MED)
3	Speed HIGH (GET /SPD/HIGH)
Q	Quit (sends stop first)

This path bypasses all internet-facing components, reducing round-trip latency to a single LAN HTTP hop (<5 ms typical).

4.6 Modes of Operation

Mode	Input	Network Path
Voice	Chrome / Edge browser	Browser → Vercel → ngrok → FastAPI → ESP32
D-pad	Any browser / device	Browser → Vercel → ngrok → FastAPI → ESP32
Keyboard	Terminal (WASD + 1/2/3)	Python script → ESP32 direct (LAN only, no ngrok)
Speed btn	Browser buttons	Browser → Vercel → ngrok → FastAPI → ESP32
Speed voice	<i>faster / slower</i>	Same path as voice

Table 4.1: Summary of control modes and their network paths.

Chapter 5

Results and Discussion

The system was successfully demonstrated on physical hardware. The following performance characteristics and observations were recorded.

5.1 Latency Measurements

- **Voice-to-motor latency:** 200–400 ms from utterance to wheel movement. This is dominated by speech recognition finalisation (≈ 150 –300 ms) rather than the network round-trip, as the Web Speech API buffers audio before emitting a final result.
- **API round-trip time:** 180–250 ms average for the full HTTPS round trip (Vercel $\rightarrow$ ngrok $\rightarrow$ FastAPI $\rightarrow$ ESP32 $\rightarrow$ return) on a home broadband connection.
- **Keyboard controller latency:** < 5 ms for a direct LAN HTTP request to the ESP32, confirming that internet-facing components contribute the majority of the overall latency.

5.2 Challenges and Solutions

- **CORS policy enforcement:** The Vercel platform generates two distinct URLs per project — a stable production URL (`ee427-robot.vercel.app`) and a deployment-specific preview URL. Both must be added to `CORS_ORIGINS` in the backend `.env` file. Additionally, trailing slashes in origin strings cause CORS mismatch failures, as the comparison is exact-string.
- **ESP32 read timeouts:** The ESP32 firmware occasionally delayed HTTP responses while executing motor commands. The bridge server was updated to distinguish read timeouts (command executed, no response body) from connect timeouts (ESP32 unreachable). Read timeouts return `{"ok":true, "esp_warning":"read_timeout"}` to prevent false 502 errors appearing in the frontend.
- **ngrok subdomain rotation:** The free ngrok tier assigns a new random subdomain on each tunnel restart, requiring a `VITE_API_URL` update and Vercel redeploy. Registering a static ngrok domain (available free with account verification) eliminates this step.

- **Voice recognition deduplication:** The initial implementation used the transcript string as a deduplication key, causing the same command to fire only once even after a pause. This was replaced with a time-based debounce (800 ms window), allowing natural repeated commands while filtering rapid duplicates from continuous recognition.

5.3 Speed Control Results

The three PWM duty cycle levels produced clearly distinguishable mechanical speeds:

- **LOW** (100/255, $\approx 39\%$): Precise slow-speed manoeuvring; suitable for indoor navigation in confined spaces.
- **MED** (175/255, $\approx 69\%$): Default operating speed; good balance of control and responsiveness.
- **HIGH** (255/255, 100%): Maximum speed; suitable for open spaces but reduced control precision.

Chapter 6

Conclusion

This project successfully demonstrates the design and implementation of an IoT-based remote control system for a four-wheeled robot car using the ESP32 microcontroller, bridging a private LAN device to the public internet through a FastAPI bridge server and ngrok reverse tunnel. Three independent control modes — voice recognition, graphical D-pad, and keyboard controller — were implemented and tested on real hardware, all converging on the same REST HTTP API exposed by the ESP32 firmware.

The architecture addresses real-world IoT deployment challenges: API key authentication for security, CORS policy configuration for cross-origin browser access, graceful handling of ESP32 read timeouts, and NAT traversal without router configuration through reverse tunnelling. The system complies with modern IoT standards by using HTTPS for all internet-facing communication, structured JSON payloads, and environment-variable-based configuration.

The project demonstrates the full network stack in practice: REST API design at the application layer, TLS 1.3 encryption at the presentation layer, persistent TCP session management at the transport layer, IP routing and ARP at the network and data-link layers, and 802.11 Wi-Fi radio at the physical layer. The modular three-tier architecture (ESP32 firmware / FastAPI bridge / React frontend) is directly extensible to other IoT control scenarios, including the ceiling fan speed controller described in the project specification.

Bibliography

- [1] Espressif Systems, *ESP32 Technical Reference Manual*, Espressif Systems, 2023. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf
- [2] Espressif Systems, *Arduino Core for the ESP32*, 2024. [Online]. Available: <https://docs.espressif.com/projects/arduino-esp32/en/latest/>. Accessed: April 2025.
- [3] S. Ramirez, *FastAPI Documentation*, 2024. [Online]. Available: <https://fastapi.tiangolo.com/>. Accessed: April 2025.
- [4] MDN Web Docs, *Web Speech API*, Mozilla Developer Network, 2024. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Web_Speech_API. Accessed: April 2025.
- [5] ngrok Inc., *ngrok Documentation*, 2024. [Online]. Available: <https://ngrok.com/docs>. Accessed: April 2025.
- [6] Vercel Inc., *Vercel Documentation*, 2024. [Online]. Available: <https://vercel.com/docs>. Accessed: April 2025.
- [7] R. Fielding and J. Reschke, *RFC 7231: Hypertext Transfer Protocol (HTTP/1.1): Semantics and Content*, IETF, 2014. [Online]. Available: <https://tools.ietf.org/html/rfc7231>
- [8] E. Rescorla, *RFC 8446: The Transport Layer Security (TLS) Protocol Version 1.3*, IETF, 2018. [Online]. Available: <https://tools.ietf.org/html/rfc8446>

Source Code

Listing 1: ESP32 Firmware (Arduino C++)

```
1 #include <WiFi.h>
2 #include <WebServer.h>
3
4 const char* WIFI_SSID      = "YOUR_SSID";
5 const char* WIFI_PASSWORD = "YOUR_PASSWORD";
6
7 // L298N pins
8 #define IN1 12    // Left motor direction A
9 #define IN2 13    // Left motor direction B
10 #define IN3 25    // Right motor direction A
11 #define IN4 26    // Right motor direction B
12 #define ENA 14    // Left PWM (LEDC channel 0)
13 #define ENB 27    // Right PWM (LEDC channel 1)
14
15 const int PWM_FREQ = 1000;
16 const int PWM_RES  = 8;    // 0-255
17 int      gSpeed    = 175;  // default: MED
18
19 WebServer server(80);
20
21 void setMotor(int a, int b, int c, int d, int spd) {
22     digitalWrite(IN1,a); digitalWrite(IN2,b);
23     digitalWrite(IN3,c); digitalWrite(IN4,d);
24     ledcWrite(0,spd);    ledcWrite(1,spd);
25 }
26
27 void handleForward() { setMotor(1,0,1,0,gSpeed);
28                       server.send(200,"text/plain","F"); }
29 void handleBackward() { setMotor(0,1,0,1,gSpeed);
30                       server.send(200,"text/plain","B"); }
31 void handleLeft()     { setMotor(0,1,1,0,gSpeed);
32                       server.send(200,"text/plain","L"); }
33 void handleRight()    { setMotor(1,0,0,1,gSpeed);
34                       server.send(200,"text/plain","R"); }
35 void handleStop()     { setMotor(0,0,0,0,0);
36                       server.send(200,"text/plain","S"); }
37 void handleSpdLow()   { gSpeed=100;
38                       server.send(200,"text/plain","SPD:LOW"); }
39 void handleSpdMed()   { gSpeed=175;
40                       server.send(200,"text/plain","SPD:MED"); }
41 void handleSpdHigh()  { gSpeed=255;
42                       server.send(200,"text/plain","SPD:HIGH"); }
43
```

```

44 void setup() {
45   Serial.begin(115200);
46   pinMode(IN1,OUTPUT); pinMode(IN2,OUTPUT);
47   pinMode(IN3,OUTPUT); pinMode(IN4,OUTPUT);
48   ledcSetup(0,PWM_FREQ,PWM_RES); ledcAttachPin(ENA,0);
49   ledcSetup(1,PWM_FREQ,PWM_RES); ledcAttachPin(ENB,1);
50
51   WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
52   while (WiFi.status() != WL_CONNECTED){delay(500);Serial.print(".");}
53   Serial.println("\nConnected. IP: "+WiFi.localIP().toString());
54
55   server.on("/F",      handleForward);
56   server.on("/B",      handleBackward);
57   server.on("/L",      handleLeft);
58   server.on("/R",      handleRight);
59   server.on("/S",      handleStop);
60   server.on("/SPD/LOW", handleSpdLow);
61   server.on("/SPD/MED", handleSpdMed);
62   server.on("/SPD/HIGH", handleSpdHigh);
63   server.begin();
64   Serial.println("HTTP server started on port 80");
65 }
66
67 void loop() { server.handleClient(); }

```

Listing 6.1: ESP32 firmware — Wi-Fi HTTP server with motion and speed control.

Listing 2: FastAPI Bridge Server (Python)

```

1  import logging, os
2  from pathlib import Path
3  from typing import Literal
4
5  import requests
6  from dotenv import load_dotenv
7  from fastapi import FastAPI, Header, HTTPException
8  from fastapi.middleware.cors import CORSMiddleware
9  from pydantic import BaseModel, Field
10
11  load_dotenv(Path(__file__).resolve().parent.parent / ".env")
12  logger = logging.getLogger(__name__)
13
14  _esp_session = requests.Session()
15  _esp_session.headers.update({"Connection": "keep-alive"})
16
17  CMD_TO_PATH = {"forward": "F", "backward": "B",
18               "left": "L", "right": "R", "stop": "S"}
19  SPEED_TO_PATH = {"LOW": "SPD/LOW", "MED": "SPD/MED", "HIGH": "SPD/HIGH"}
20
21  def _env(name, default=None):
22     v = os.environ.get(name)
23     return v if v not in (None, "") else default
24
25  def _esp_base():
26     return (_env("ESP32_BASE_URL", "http://192.168.1.192") or "").rstrip(
        ("/")
    )

```

```

27
28 def _check_api_key(key):
29     expected = _env("BRIDGE_API_KEY")
30     if not expected: raise HTTPException(500, "BRIDGE_API_KEY not set")
31     if key != expected: raise HTTPException(401, "Invalid API key")
32
33 def _esp_get(path):
34     url = f"{_esp_base()}/{path.lstrip('/')}"
35     try:
36         r = _esp_session.get(url, timeout=(
37             float(_env("ESP32_HTTP_CONNECT_TIMEOUT", "2")),
38             float(_env("ESP32_HTTP_READ_TIMEOUT", "12"))))
39         return r, False
40     except requests.exceptions.ReadTimeout:
41         return None, True # command ran; ESP32 slow to respond
42     except requests.exceptions.RequestException as e:
43         raise HTTPException(502, f"Could not reach ESP32: {e}") from e
44
45 class CommandBody(BaseModel):
46     cmd: Literal["forward", "backward", "left", "right", "stop"]
47
48 class SpeedBody(BaseModel):
49     speed: Literal["LOW", "MED", "HIGH"]
50
51 app = FastAPI(title="Robot Bridge")
52 app.add_middleware(CORSMiddleware,
53     allow_origins=[o.strip() for o in
54         (_env("CORS_ORIGINS", "http://localhost:5173") or "").split(",")
55     ],
56     allow_methods=["GET", "POST", "OPTIONS"], allow_headers=["*"])
57
58 @app.get("/health")
59 def health(): return {"status": "ok"}
60
61 @app.post("/api/command")
62 def api_command(body: CommandBody,
63     x_api_key: str|None = Header(default=None, alias="X-API-Key")):
64     _check_api_key(x_api_key)
65     r, slow = _esp_get(CMD_TO_PATH[body.cmd])
66     if slow:
67         return {"ok": True, "cmd": body.cmd, "esp_warning": "read_timeout"}
68     if r.status_code != 200:
69         raise HTTPException(502, f"ESP32 returned {r.status_code}")
70     return {"ok": True, "cmd": body.cmd}
71
72 @app.post("/api/speed")
73 def api_speed(body: SpeedBody,
74     x_api_key: str|None = Header(default=None, alias="X-API-Key")):
75     _check_api_key(x_api_key)
76     r, slow = _esp_get(SPEED_TO_PATH[body.speed])
77     if slow:
78         return {"ok": True, "speed": body.speed, "esp_warning": "read_timeout"}
79     if r.status_code != 200:
80         raise HTTPException(502, f"ESP32 returned {r.status_code}")
81     return {"ok": True, "speed": body.speed}

```

Listing 6.2: FastAPI bridge — authenticates and forwards commands to the ESP32.

Listing 3: Voice Command Matching (TypeScript)

```
1 export type DriveCmd    = 'forward' | 'backward' | 'left' | 'right' | 'stop'
2 export type SpeedCmd    = 'speed_up' | 'speed_down'
3 export type VoiceCmd    = DriveCmd | SpeedCmd
4 export type SpeedLevel = 'LOW' | 'MED' | 'HIGH'
5 export const SPEED_ORDER: SpeedLevel[] = ['LOW', 'MED', 'HIGH']
6
7 export function transcriptToCommand(text: string): VoiceCmd | null {
8     const t = text.toLowerCase()
9     if (/^stop$/.test(t))           return 'stop'
10    if (/^backward$/.test(t))       return 'backward'
11    if (/^left$/.test(t))           return 'left'
12    if (/^right$/.test(t))          return 'right'
13    if (/^forward$/.test(t))        return 'forward'
14    if (/^faster$/.test(t))         return 'speed_up'
15    if (/^slower$/.test(t))         return 'speed_down'
16    return null
17 }
```

Listing 6.3: commands.ts — maps speech transcript to drive and speed commands.